A Systematic Approach to Process Enactment Analysis as Input to Software Process Improvement or Tailoring

Ming Huo, He Zhang, Ross Jeffery

National ICT Australia Ltd. and University of New South Wales, Australia

{mhuo, hezhang, rossj}@cse.unsw.edu.au

Abstract

Software process improvement has been a focus of industry for many years. To assist the procedure and implementation of process improvement we provide a software process recovery method based on mining project enactment data. The goal of the method is to uncover the actual process used in order to provide input to improve the quality of a defined software process. The recovered model (or patterns) is at the same level of abstraction as the predefined process model. This provides an easy and clear way to identify the gap between the planned process model and the real enactment. We investigate the enactment of a defined software process from the view of understanding the appropriateness and fitness for purpose of the process model from the viewpoint of the project managers in the context of a small software development organization. We collected data from organizations and applied our method to a pilot case study. The main contribution of our work is to provide a software process model recovery method which supports software process change and improvement.

1. Introduction

One implicit assumption in software process research is that improving the software process will improve the software product quality, and better control of the software process will increase project success. Hence software process improvement has been a long term pursuit. Practitioners from research areas as well as industry have continued to explore techniques to improve software processes. However software process has long been considered as a complex procedure [4]. Software process models and software process modeling languages have been developed over many years. Practitioners are curious as to what is really happening in the enacted process, such as if the process model is applicable for the project and how appropriate it is, and if there is a gap between the pre-defined process and the project enactment? In order to answer these questions, our work attempts to discover the variance between the planned process model and its enactment by using process mining.

In section 2, we outline previous work in process mining. Section 3 describes the procedure and preliminary notations and algorithm involved. Section 4 presents our case study of one project with data collected from a software company, as well as the problems identified. It is followed by comparing our approach with previous work and discussion as well as some limitations found in the case study. The final section is the conclusions and future work.

2. Related work

Process mining intends to recover a process model from people's daily activities. The idea of process mining is not new but was mainly developed in the business process area, especially in workflow management systems [12]. Cook and Wolf have done a similar investigation in the software process field [3]. There are two major problems when utilizing them to recover a software process model. The first problem is that hundreds of process instance data have to be collected to recover a process model because they treat each event or task as the smallest unit in a process instance [1, 9, 11] and one process contains hundreds of events. Different projects may follow different pre-defined process models. Hence it is very difficult to collect sufficient data of many projects that consistently follow one process model. In this context, the data might never be enough for recovering one process model. The second problem is the recovered model is a low-level process model defined generally in the terms of daily activities. Most pre-defined software process models, such as the waterfall model, are moderately high-level process models which are very difficult to compare with the activity level models. There is a distinct gap between the low level recovered model and the pre-defined higher-level model.

In this paper, we present an approach that overcomes these two problems. We begin by collecting low-level project task data, which is then mined for process patterns by analyzing these data. The recovered

XIII ASIA PACIFIC SOFTWARE ENGINEERING CONFERENCE (APSEC'06)
0-7695-2685-3/06/$20.00 © 2006 IEEE
Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 13,2026 at 01:57:12 UTC from IEEE Xplore. Restrictions apply.

process patterns are at the same level as the pre-defined process model. It is easy to discover any discrepancies between them, and by analysis, to improve the pre-defined process through empirically based tailoring.

3. Process recovery method

This section presents our process recovering approach, the primary algorithm and notations. Firstly, we give a brief overview and assumptions of our approach, and then followed by the data analysis methods, and the process patterns mining approach.

We assume that a set of the process models or methods, such as waterfall or ISO, has been selected by the software company. The company will normally plan the process first, and then execute the process. Meanwhile, the company will also design and tailor the process according to their environment. A software process consists of a set of sequenced tasks leading to design, development or maintenance of a software product [3]. It is related to many factors, e.g. company environment, product and project type. Our work focuses on the behavior aspect of the process. We investigate the deviations between the pre-defined process and the enactment pursued by the development team. The process model (or patterns) we are identifying from the project enactment data is a high-level process model when compared with previous work (explained in Section 2). Figure 3-1 shows the four major steps in our approach and each step is discussed in the following sections.

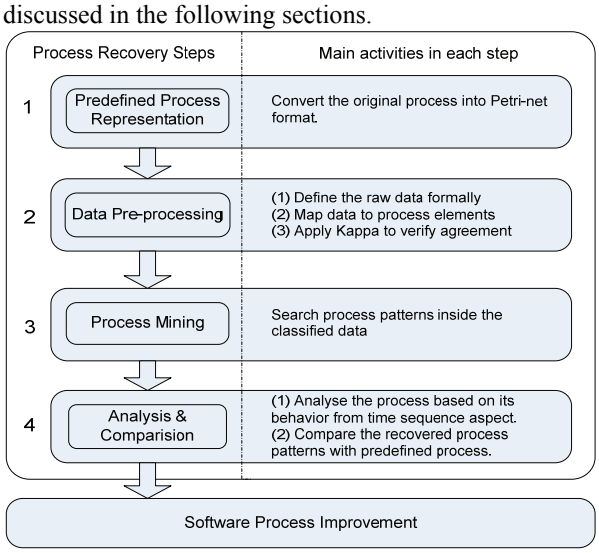


Figure 3-1 Process recovery approach

3.1 Process representation

Before we introduce our data analysis method, we transform a defined process model into a formal format. Petri-net has been extensively used as a process modeling language due to its formal definition and graphical characteristic [10]. We chose Petri-net to represent a process model for two reasons. The first is for the easy comparison of recovered process patterns and a predefined process model. The outputs of process mining are process patterns in Petri-net format. The second reason is that across different organizations a process model may be presented in various formats and Petri-nets provide a unified format. Below we describe the basic assumptions and notations of a process model:

1. After a process model has been established in one organization or one development team, the team members will follow it in their projects.
2. A process consists of a set of sequenced steps/tasks which we call process elements.
3. There are relationships among process elements, e.g. software design starts before coding.
4. There should be a set of well-defined steps in a process model and each step, i.e. a **process element,** presented as ***O***.

These process elements are sequentially connected. The relations (***E***) exist among the process elements. The path indicating the relations is called a **place, *P***. One process element may have connections to many different process elements. However, there is single connection between two elements and it is marked with a place. A process with these basic characteristics can be described formally and represented in a Petri-net format. The notations are defined below:

- ***O*** is a finite set of elements.
- ***V*** is a subset of the set O: $V = \{vi \mid vi \in O\}$
- ***E*** is a set of relations on the set V, i.e. the transitions among the process elements.
- ***P*** is the place and it indicates the paths.

At this stage, the process model will be represented in Petri-net format. A net as $N = (O, P, E)$.

3.2 Process data hierarchy

In this section, the data hierarchy status is portrayed and the data type is highlighted. In one project, there might be different types of data including high level and low level data. The low level data may include project tasks, daily activities and etc. The data we process are the low level data gathered during a process model implementing in projects. In this circumstance, the data reflect how the team followed the process model in the real world. We lay out the possible data hierarchy normally of a software project.

Data hierarchy: The high level data are items such as project documents and system requirements which are used as reference data. A set of project ***tasks*** is designed according to the process model. A ***task*** contains many ***activities***. Therefore, the smallest unit data is one ***activity***. A task is the second smallest data

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 13,2026 at 01:57:12 UTC from IEEE Xplore. Restrictions apply.

unit. The hierarchy of all the data is shown in Figure 3-2. In this approach, we focus mainly on the task level. We define the data in a common formal method. We assume it is possible to record tasks such that:

1. Each task refers to only one process instance.
2. Tasks are ordered and it has a finite period time from start to end. (some tasks are executed in parallel or overlapped)
3. Task refers to the process model.
4. Each task has a description.
5. Each task has one or more assignees, the person who executed the task.

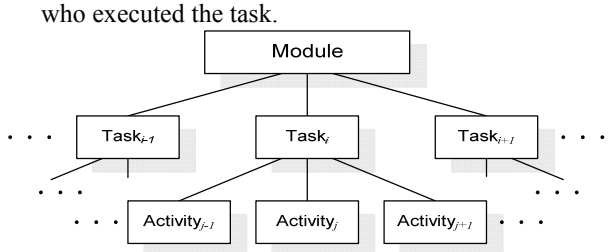


Figure 3-2 Process data hierarchy

3.3 Process data pre-processing

The data needs pretreatment before process mining. There are three steps in the data pretreatment, 1) formalize the data, 2) classify the data according to process elements, and 3) verify the data classification. We are expecting our recovered process patterns or model to be at the same level as the predefined process model. However, the collected raw data are at the low level and process mining is used to search the process patterns in the low level data. In order to reflect the relationships of the process elements, we need to classify the data according to the process elements. To ensure the classification quality and to evaluate the agreement degree, Kappa statistic method is used to describe the strength of agreement in step three [7].

Data formalization: Given a task $\boldsymbol{t}$, $\boldsymbol{Start(t)}$ represents the start date of t and $End(t)$ represents the end date of $\boldsymbol{t}$. Obviously, $\boldsymbol{End(t) > Start(t)}$. We use $\boldsymbol{V_t(t_i)}$ to represent the process elements that task $\boldsymbol{t_i}$ belongs to: $\boldsymbol{V_t(t_i) = \{t_i | \ t_i \in T(v_i)\}}$

Data classification: In this step, we need to map tasks to process element in $\boldsymbol{O}$. It is one of the important aspects in our approach. One reason is that task classification can only be done manually since these tasks are fairly low level data with some detailed information, e.g. start time, comments and etc. The other reason is that it is difficult to categorize the tasks to the process elements, because only one project can be processed at one time which also involves process experts. It costs human resource and consumes time.

The raters need to possess comprehensive knowledge of the predefined process model. They also need to understand the project content. In addition, a list of the task descriptions is provided to the raters. They do the classification individually.

Verification of task classification: Kappa is a chance-corrected measure of agreement among raters, each of whom independently classifies each sample of subjects into one of a set of mutually exclusive and exhaustive categories. Its value is computed as (3-1).

$$K = \frac{p_o - p_e}{1 - p_e} \qquad (3\text{-}1)$$

P_o is observed agreement ; P_e is expected agreement:

$$p_o = \sum_{i=1}^{k} p_{ii} \ , \quad p_e = \sum_{i=1}^{k} p_i^1 p_i^2$$

$K=0$ means that agreement is not different from chance and $K=1$ means perfect agreement. The conclusion can be drawn when K value is above 0.8 [8]. Our approach won't accept the classification till K value above 0.8.

3.4 Process patterns mining

There are many process mining algorithms. Alpha algorithm is the basic one which we borrow the main idea from, including generating temporal relations and connecting transitions [11].

After each task is assigned into one element $\boldsymbol{v_i}$ in $\boldsymbol{O}$. We use $\boldsymbol{T(v_i)}$ to represent all the tasks that are assigned into element $\boldsymbol{v_i}$. The next step is to mine the data. We focus on the temporal relations to investigate if there are two tasks with temporal dependency. For example, if one task is always followed by another task, there is likely a temporal relation between them. Furthermore, if such dependency exists, it implies that a connecting transition exits. There would be a place connecting the transitions[11]. While accumulating these relationships and the connecting places, the recovered process patterns are generated piece by piece.

Given two tasks t_i, t_j we say t_i and t_j have temporal relation $t_i \rightarrow t_j$ iff $End(t_i) > Start(t_j)$. We already know that each task belongs to a correspondent process element $V_t(t_i)$, which means the temporal relationship in the task level also represents the relationship in the predefined process model level. There are two distinctive differences between this method and alpha algorithm. The alpha algorithm tries to recover the whole process model and requires the data set must be complete. A complete data set means all tasks directly follow each other, which is not realistic in most of our situations. In our approach, we allow the task data set incomplete and try to find out any piece of the process patterns. When the data set is incomplete, process patterns are mined out instead of a whole process model. The second difference is that the process pattern found is at the same level as pre-defined

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 13,2026 at 01:57:12 UTC from IEEE Xplore. Restrictions apply.

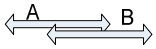
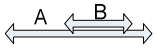
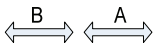
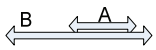
process model. Because the tasks have been mapped to the pre-defined process elements, the relations between tasks represent the relationship of the two process elements that the tasks belong to. For example, if a relation *A->B* exists between task *A* and *B*, and $V_t(A)= E_i$, $V_t(B) = E_j$ then in the recovered process, the possible path is $E_i \text{ -> } E_j$..

By recovering the relationship of tasks, the relationship of process elements are recovered from the enactment data. Then, we can easily compare the recovered process pattern and further bridge the gap with the predefined process model.

3.5 Measurement of consistence

This section presents a metric for measuring the deviation between the predefined process model and the mined process model/patterns. We try to measure the gap between the predefined process model and the recovered model. In our approach, we allow multiple paths from one process element to another and propose our own metric. The metric is time stamp conflict.

Table 3-1 Time stamp conflict conditions

	Sequence Name	Conflict type
	1) A is followed by B	None
	2) A is right overlapping with B	Right overlapping
	3) A is containing B	Containing
	4) A is exceeded by B	Sequence conflict
	5) A is left overlapped by B	Left overlapping
	6) A is contained by B	Contained

Time stamp conflict aims to measure the paths that have not been properly followed among the process elements. The development team may not properly follow the process elements of the designed sequence. There might be six possible situations listed in Table 3-1. Assume in the predefined process model, the process element that task A belongs to should be finished before the element that B belongs to. Then except the first situation, all the rest five cases are recognized as conflict with the process model. The basic assumption is that the start time is earlier than the end time, i.e. $Start(t) < End(t)$. $V_t(t_A)$ should be finished before $V_t(t_B)$, which means for $\forall\, t_1,t_2,\ t_1 \in T(V_t(t_A)),\ t_2 \in T(V_t(t_B))$, we have $End(t_1) < Start(t_2)$.

4. Case study

In this section, we describe the case study done by using our method and the results from analysis. We won't enumerate all the details of each step. Instead, we will present the values and a pattern we have discovered from this project and the potential problems found from comparing the mined process pattern with the predefined process model.

4.1 Background

An Australian software firm, which we have cooperated with for many years, chose ISO 12207 as their fundamental process model. ISO 12207 Software Life Cycle Process Standard defines three primary types of processes and two assisting processes. The company customized ISO 12207 and installed a web-accessible electronic process guide (EPG).

The EPG provides an efficient way of presenting software process models to the development team. It visualizes the process model as the graphic format with UML notations and provides documentations of each process element. It also shows the sequence of all the elements. The development team can easily follow the process model and clearly understand the content of each element with the assistance of EPG[5]. The process enactment data of many projects are stored through the EPG. Typical task information includes:

1) task start and end date,
2) assignee, the person who executes it,
3) brief comments of the task made by the assignee,
4) and which project this task belongs to.

We randomly chose one small to mid-sized project with desired documentations as our pilot case study, a project of database system development. The comprehensive documents assisted the experts in mapping the tasks to the EPG process elements. The brief project information is shown in Table 4-1.

Table 4-1 Pilot Project Information

Project Information		
Team size		6 persons
Total Effort		1652.25 man-hours
Duration		875 days
Total task number		85
Task Classification	EPG Tasks	53 (62%)
	None-EPG Tasks	32 (38%)
Kappa Agreement		0.808

4.2 Process re-presentation

We represent the EPG in formal presentation by turning it into Petri-net format. Part of the EPG Petri-net model is shown in Figure 4-1. A rectangle represents a process element which corresponds to an activity in UML notation. A circle means a place where two process elements are connected and a line with arrow means there is a relation between the elements.

- $P = \{P1, P2...P10...\}$

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 13,2026 at 01:57:12 UTC from IEEE Xplore. Restrictions apply.

- O = {"Collect Requirement", "Refine System Requirement", …, "Test", …}
- $E \subseteq (P \times O) \cup (O \times P)$

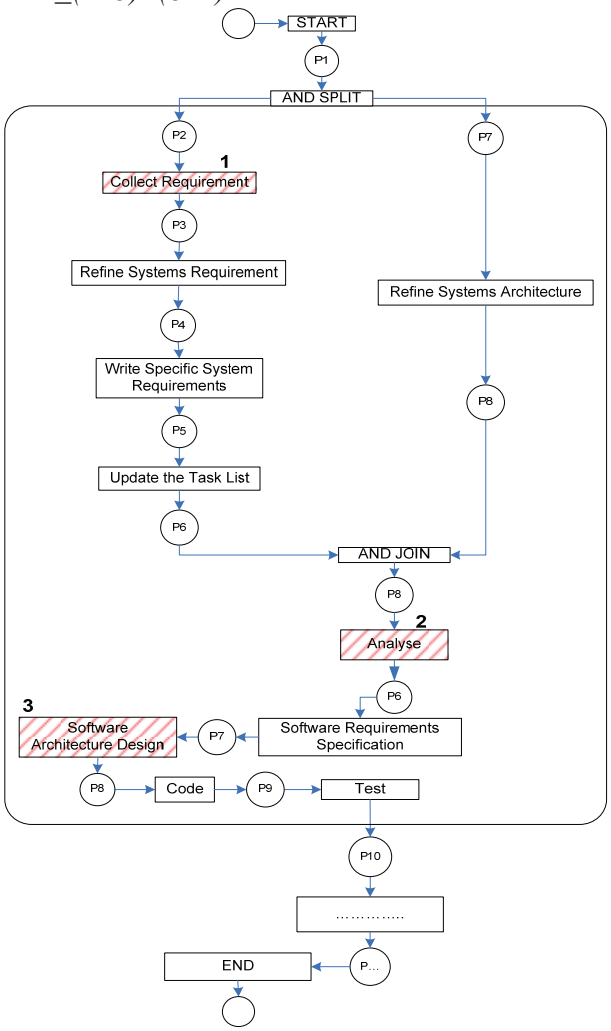


Figure 4-1 EPG Petri-net

"AND SPLIT" implies that some process elements can be done at the same time. "AND JOIN" clarify that all the process elements before it need to be finished before stepping into next process element behind it. We do not place the whole EPG Petri-net diagram in this paper. Instead, we centralize on the part where the process patterns have been identified.

4.3 Process data pre-processing

The tasks classification is done by four raters. Three of them are process experts with the comprehensive knowledge of the process model and the deep understanding of the EPG. One of the experts has worked with the company for over one year and the development team she was involved in is the team of this project. Thus, there are three levels of the raters:

- Level 1: the expert with working experience in the organization and knowledge of process model,
- Level 2: the experts in process modeling area,
- Level 3: the rater who only has basic understanding on process model.

We provide all raters the whole set of the project documents to ensure their understanding of the project content. They tried to map each task to one process element individually. During the classification, the raters found some tasks cannot be categorized to any of the process elements properly and at this stage, we put these tasks aside.

In this step, we found experts with practical knowledge of the predefined process model can reach a high agreement no matter whether they have working experience with the company. The deviation of the judgment between the novice (rater from level 3) and the process experts (level 1 and 2) is distinct. High agreement is reached by the raters from level 1 and 2. The Kappa value after correction is ***0.808***, which indicates that conclusions can be reasonably drawn on the tasks classification.

4.4 Discoveries from sequence analysis

Below we concentrate on illustration of the findings from our case study and explain them from the time stamp point of view. After the tasks were classified, we firstly measure the time sequence conflicts. There are two metrics: 1) the amount of conflicts (Figure 4-2) and 2) the number of conflict types (Figure 4-3).

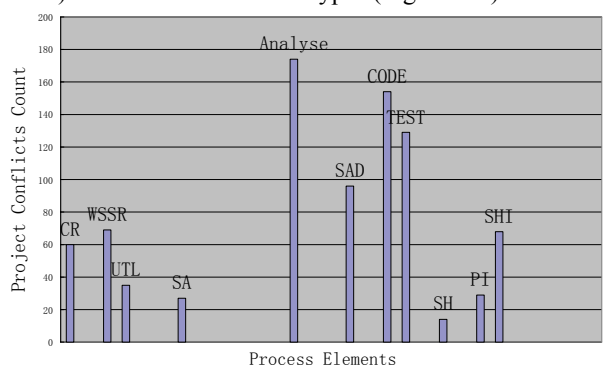


Figure 4-2 Time sequence conflicts count

Figure 4-2 shows the distinctive process elements which contain high conflicts are "Analyse", "Code", and "Test". After a deeper analysis, we investigate the conflict types in these three process elements. Figure 4-3 shows the distribution of conflict types (explained in Section 3.5) in "Analyse". It indicates that some of "Analyse" tasks happened behind its subsequence.

"Code" and "Test", also have a large number of conflicts. Figure 4-4 shows the conflict types in them. The "Sequence" and "Containing" have dominant amount conflicts. Then we have a further look at

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 13,2026 at 01:57:12 UTC from IEEE Xplore. Restrictions apply.

"Code" and "Test". In the "Code" element, "Analyse" and "Test" are the top two process elements that have sequence conflicts, which mean "Code" jumped prior to some "Analyse" actions and some "Test" actions happened before coding.

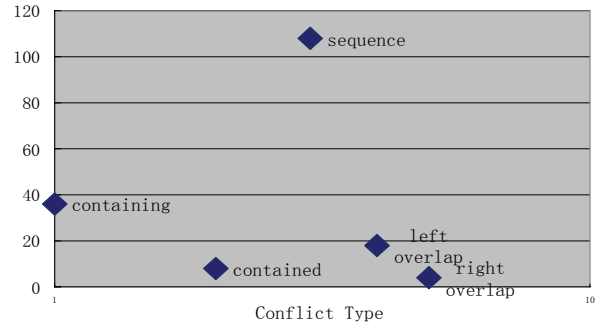


Figure 4-3 Analyse conflict type distribution

Meanwhile, "Analyse" and "Write Specific Requirements" are the top two process elements that have "containing" conflicts with "Code", which means the actions of these three process elements may happen parallel. It also indicates that the team may do some "Analyse" even after "Test". In "containing" conflict type, "Analyse" and "Collect Requirement" rank the top two, which shows that these three process elements ("Test", "Code" and "Analyse") may occur together.

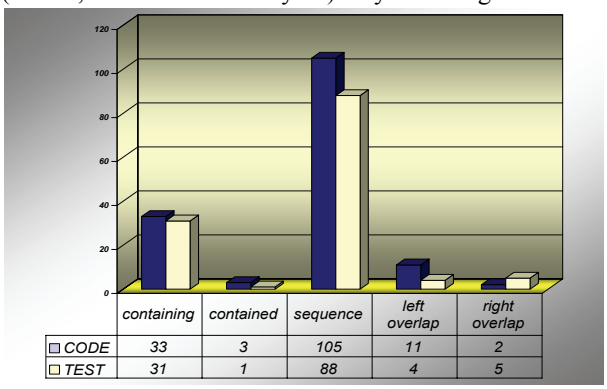


Figure 4-4 Code and Test conflicts distribution

The above phenomena imply the following situations that may happen during the development.

1. Test and code tightly cooperate with each other and there is a short cycle of Code-Test-Code.
2. New requirements may emerge at any time because "Collect Requirement" even happened in testing.
3. New requirements may be added to the project at any time during the development which causes the action of "Analyse" occurring in any phase.

4.5 Discoveries from process mining

During the process mining, we found the case project data set is incomplete and a whole process model cannot be recovered. Instead, we found some partial patterns. The patterns recovered by mining the data are shown in Figure 4-5. In Figure 4-1, "Collect Requirement" (CR) should be done at the very early stage and then followed by "Refine System Requirements" (RSR). There are 6 places, from P2 to P7, in the EPG. However, in the recovered pattern (Figure 4-5), there is only one place between them, i.e. five process elements have been skipped. It forms a loop, from CR to "Analyse", which indicates that they are tightly cooperated. It may reveal the following phenomena from two aspects: 1) the requirements may be unstable during the development, even after the first gathering of some requirements, the team found they still need to keep collecting or mending requirements due to client's request; 2) the team may not be able to collect accurate requirements at the beginning to generate well defined software architecture.

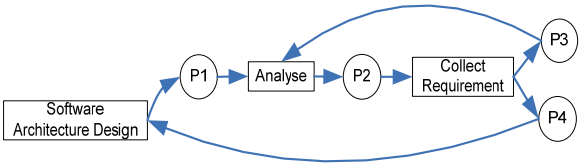


Figure 4-5 Pilot Case Study Result

The direct connection from "Collect Requirement" to "Software Architecture Design" shows that the team skipped "Specific System Requirements" and the "Refine System Requirements" tasks. Hereby, some questions may be raised for the manager: 1) "Does the development really need these process elements?", 2) "Are the skipped process elements not adopted in many projects or only in this one?" or 3) "Under what circumstance these process elements can be skipped?"

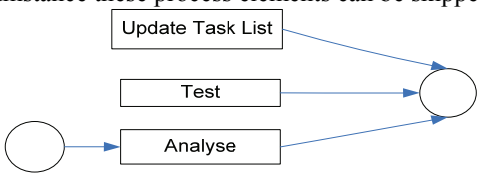


Figure 4-6 Ending Process Elements

Even though some other process elements are also found, the project looks incomplete compared with EPG. Figure 4-6 shows that the process may end at "Update Task List", "Test" or "Analyse". All three of these elements are not the proper ending elements. As defined in EPG, projects should end at "Provide Documentation and Support". We had a close look at the source data of the tasks within these process elements and found that when the project was near its end, clients still wanted to have some meetings and discussion on how to finalize the project. The clients kept sending change requests, even at the ending point.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 13,2026 at 01:57:12 UTC from IEEE Xplore. Restrictions apply.

From the above findings, we realize that there is a gap between the EPG and project enactment of this project. Further questions are raised as "Shall we revise the pre-defined process based on the discovered process patterns?" or "Shall the manager keep the team on track with the pre-defined process model?"

4.6 Other issues

There are 38% of the tasks which haven't been classified into the predefined process model. These tasks reveal that the scope of the process model cannot fully cover the designed tasks. Although we cannot arbitrarily claim that the process model needs to be changed, we need to consider the situation from multiple viewpoints (discussed in section 5.2).

The case study indicates that the process model is not sufficient. More process elements need to be introduced based on the previous analysis and the scope of the process model needs to be extended.

5. Discussion

In this section, we compare our work with previous related work from three aspects: 1) data type and size, 2) mining method, and 3) output. Process improvement is also discussed from multiple aspects. Finally, we discuss some limitations of our approach.

5.1 Comparison with previous work

Data type and required amount: the previous work used a similar approach in the software process context. Their approach involved statistical methods where the amount of data required is large, normally more than hundreds of cases. It is difficult to gather hundreds of project enactment data which follow the same process model within the similar project environment. The required data size of our approach could be fairly small.

Mining method: Most of the process mining approaches involve statistical methods. Instead, our approach focuses on a process mining algorithm. We try to find the temporal relations from the data set and then to detect the connecting places from the relations. These places made up the recovered process patterns.

Output: The output of previous work presents the relationships among the daily activities, such as "code check-in", "code inspection" and etc. The input data for process mining is at the lowest level. Compared with our data hierarchy (Figure 3-2), their data stays at the "activity" level. Without mapping these data to process elements, the recovered model with such details is normally very large and mainly reflects the low-level process model. In addition, the transitions may be countless and complex. The recovered process model or patterns through our method are in the same context as the pre-defined process model. The major advantage is that a high-level process pattern or model can be captured. This is the most distinctive difference with the previous work.

5.2 Process improvement

This part shows how our approach contributes to process improvement from three points of view. The results generated from our approach can contribute to: 1) process improvement, 2) process evolution, and 3) process tailoring. In each aspect, we first present our assumptions and then show how our results contribute to each of them. Software process involves many factors such as development, team capability and productivity, organization environment, and the project characteristics and so on. An effective software process should be able to model the behavior of the team accomplishing one real project [13]. Our common assumptions are: 1) SPI needs process assessment and its results can be as input for the improvement plan. Comparing with process evolution, its changed content is larger and the speed is slower; 2) the degree of process amendment in process evolution is deeper than process tailoring and it may happen in different domains such as organization environment and development method; 3) in software tailoring, the core part is stable and the changes are less than the other two aspects.

Process improvement: we assume that a perfect process model can be reached by improvement in an organization. At the final stage, a stable process model should be developed that can cope with any circumstances. Software process assessment is the critical step when implementing SPI and is valuable only when the difference between the planned process model and executed model is discovered and measured. In our approach, the process is diagnosed and tracked, and then the gap is identified. The results through our approach can be input to process assessment.

Software process evolution: Software process evolution is treated as continuous process improvement [6]. We assume that a perfect process model does not exist in an organization and the process model needs to keep changing and there is no ending status. Normally new rules and changes are established based on problems met during the execution and they are injected into the original process model. The analysis of the enactment process patterns from our approach points out the problems from the manager's view directly and generates questions for the project leaders.

A need of process tailoring: The assumption is there is a core predefined process model and it needs to be tailored for each specific project according to the project's context. The core predefined process model should consist of the fundamental process elements.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 13,2026 at 01:57:12 UTC from IEEE Xplore. Restrictions apply.

Tailoring a process model to fit a specific condition is presented in [2]. Process patterns extracted from the enactment data can assist the managers in selecting process activities or tasks during tailoring. If the manager can be given with more project data, such as process performance, ROI, he can accurately create the tailoring rules with our result. These patterns can also be added to an experience repository and contribute to establish guidelines for future process tailoring.

5.3 Limitations

From the pilot case study, we find some issues needs to be improved in our future work. The first issue is that our approach does not handle non-process element tasks. While doing the task classification step in the pilot case study, some tasks cannot be clearly categorized to one single process element. Meanwhile some tasks do not belong to any process elements at all. Currently our approach is unable to handle these data. The input data for process mining are "pure" tasks, which can be surely identified to one process element.

The second problem is that while the project is a moderate-large sized project, it needs to be divided into its modules or components. At this stage, our approach has no clearly instruction on what type of projects should be divided into modules.

6. Conclusion and future work

In this paper, we presented our approach to recover process patterns based on project tasks. In our method, process patterns can be discovered in the same format as a predefined process model, which can be designed from any general process methods, e.g. waterfall model. In the previous research, the recovered process remains at the low level, in other word, the same level as the raw data.

The breakthrough of our work is that we can discover higher-level process patterns or model at the same level as the pre-defined process. The deviations between a pre-defined process model and the discovered process patterns can be taken as the input for software process improvement, evolution or tailoring. The value of our method can be summarized as: 1) Provide evidence of how a process executes during the development; 2) Identify the coverage of the process model; 3) Extract the process patterns from live data; 4) Discover the conflicts between the defined process model and the enactment model.

In our work, we don't propose a method to analyze the tasks that cannot be mapped to any process element. This remains as future work. After mining process patterns, how to accurately compare them with the defined process model and to present the deviation quantitatively will also be investigated. Besides, a comprehensive process amendment mechanism for using these results will be proposed in the future.

7. References

1. Alves de Medeiros, A.K., A.J.M.M. Weijters, and W. van der Aalst, *Using Genetic Algorithms to Mine Process Models: Representation, Operators and Results.* 2003.
2. Basili, V.R. and Rombach, H.D. *Tailoring the software process to project goals and environments*, in *Proceedings of the 9th international conference on Software Engineering*. 1987, IEEE Computer Society Press: Monterey, California, United States. p. 345-357.
3. Cook, J., E. and A. Wolf, L., *Discovering models of software processes from event-based data.* ACM Trans. Softw. Eng. Methodol., 1998. **7**(3): p. 215-249.
4. Fuggetta, A., *Software process: a roadmap* in *Proceedings of the Conference on The Future of Software Engineering* 2000 ACM Press: Limerick, Ireland p. 25-34
5. Jeffery, R. and Kurniawati, F. *The Use and Effects of an Electronic Process Guide and Experience Reposicoty: A Longitudinal Study* Information and Software Technology, Elsevier, Amsterdam, 2005.
6. Kellner, M.I., L. Briand, and J.W. Over. *A method for designing, defining, and evolving software processes*. 1996.
7. Henningsson, K. and Wohlin, C. *Assuring Fault Classification Agreement -- An Empirical Evaluation*. in *ISESE* 2004.
8. Krippendorff, K., *Content Analysis: An Introduction to Its Methodology.* Newbury Park, CA: Sage Publications., 1980.
9. Wen, L. and W. van der Aalst, Z. Wang, and J. Sun., *A Novel Approach for Process Mining Based on Event Types.* BETA Working Paper Series, WP 118, Eindhoven University of Technology, Eindhoven, 2004.
10. Murata, T., *Petri nets: Properties, analysis and applications.* Proceedings of the IEEE, 1989. 77 (4): p. 541 - 580
11. van der Aalst, W., T. Weijters, and L. Maruster, *Workflow Mining: Discovering Process Models from Event Logs.* Knowledge and Data Engineering, IEEE Transactions 2004. 16 (9): p. 1128 - 1142.
12. van der Aalst, W. *Verification of Workflow Nets*. in *Application and Theory of Petri Nets, 18th International Conference*. 1997. Toulouse, France.
13. Watts, S.H. and I.K. Marc, *Software process modeling: principles of entity process models*, in *Proceedings of the 11th international conference on Software engineering*. 1989, ACM Press: Pittsburgh, Pennsylvania, United States.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 13,2026 at 01:57:12 UTC from IEEE Xplore. Restrictions apply.